

Unterrichtsmaterialien zum Thema

Raspberry Pi – Meteor-M Satellitenbilder

Digitaler Empfang im 137-MHz-Band (LRPT) • Headless-Betrieb über SSH • Material für SchülerInnen

Warum nicht mehr NOAA?

Die analogen NOAA-Wettersatelliten senden nicht mehr. NOAA-18 wurde am 06.06.2025 abgeschaltet, NOAA-19 am 13.08.2025 und NOAA-15 am 19.08.2025. Damit ist die Übertragungsart APT, die seit 1963 in Betrieb war, endgültig Geschichte.

Die Satelliten selbst kreisen weiter um die Erde – abgeschaltet wurden nur ihre Sender. Ältere Anleitungen führen deshalb zwangsläufig zu einem leeren Bildordner.

Ersatz bieten die russischen Meteor-M-Satelliten. Sie senden im selben Frequenzband, aber digital. Eure Antenne bleibt dieselbe – nur die Software ändert sich.

Headless – was heißt das für euch?

Der Raspberry Pi läuft ohne Monitor, Tastatur und Maus. Ihr steuert ihn ausschließlich über SSH von einem anderen Rechner aus. Das hat drei Konsequenzen:

1. Es gibt keine grafische Oberfläche. Alles läuft über die Kommandozeile – auch der Empfang.
2. Es gibt keinen Wasserfall zum Hinschauen. Ob der Empfang funktioniert, lest ihr stattdessen aus den Log-Ausgaben ab. Kapitel 6 zeigt, worauf ihr achten müsst.
3. Überflüge müssen selbst eingeplant werden. Ihr schreibt dafür zwei kleine Shell-Skripte – genau wie beim alten NOAA-Aufbau, nur kürzer und korrekt.

Materialliste

Allgemein:

- ☐ Raspberry Pi 4 oder 5 (Model B)
- ☐ Netzteil und/oder eine Powerbank
- ☐ SD-Karte (mind. 16 GB)
- ☐ Einen RTL-SDR-USB-Stick (z. B. Nooelec NESDR SMARt v4 oder RTL-SDR Blog V4)
- ☐ Empfohlen: SAW-Filter/LNA für 137 MHz (Aufdruck 1581) – deutlich bessere Bilder in Ortsnähe
- ☐ Einen zweiten Computer mit SSH-Client (Windows: PowerShell oder PuTTY; Linux/macOS: Terminal)
- ☐ Netzwerkverbindung für den Pi: LAN-Kabel oder W-LAN
- ☐ Eine konstante Internetverbindung für den Installationsprozess

Für den Bau der Antenne:

- ☐ 2x Aluminiumstange, mind. 55 cm lang & 3,4 mm Ø
- ☐ 2x Lüsterklemme
- ☐ Coaxial-Kabel mit 50 Ω & SMA-Anschluss
- ☐ Holzbrett o. ä.
- ☐ Klebeband
- ☐ Abisolierzange
- ☐ Metallsäge
- ☐ Stift & Schere

Was ihr NICHT braucht

Monitor, Tastatur und Maus für den Pi. Im Headless-Betrieb bleibt der HDMI-Anschluss leer.

Ein Raspberry Pi 3 reicht rechnerisch aus (ohne Desktop läuft auch der Empfang darauf), aber SatDump muss selbst kompiliert werden – auf einem Pi 3 dauert das mehrere Stunden und muss mit make -j1 laufen, sonst geht der Arbeitsspeicher aus.

Bauanleitung: V-Dipol-Antenne

Die Antenne ist dieselbe wie beim NOAA-Empfang – ihr könnt eine vorhandene V-Dipol-Antenne unverändert weiterverwenden. Zur Erinnerung die Rechnung, diesmal für die Meteor-Frequenz:

Meteor-M N2-4 sendet auf **137,900 MHz** (Reserve: 137,100 MHz). Daraus folgt die Wellenlänge

$$\lambda = c / f = 299\,792\,458 \text{ m/s} \div 137,9 \text{ MHz} = 2,174 \text{ m}$$

$$\text{Halbwellendipol (beide Stäbe zusammen): } \lambda/2 = 1,087 \text{ m}$$

$$\text{Ein einzelner Stab: } \lambda/4 = 0,543 \text{ m} = 54,3 \text{ cm}$$

$$\text{abzüglich ca. 1 cm für Lüsterklemme: } \approx 53,3 \text{ cm}$$

Der Unterschied zur NOAA-Antenne (53,5 cm) beträgt 2 mm und liegt damit unter eurer Sägegenauigkeit. Eine vorhandene Antenne muss nicht neu gebaut werden.

Aufbau

- 1 Schneidet ein Ende des Coaxial-Kabels ab. Legt die innere Kupferlitze sowie die äußere Kupferummantelung frei.
- 2 Sägt die Aluminiumstangen auf 53,3 cm zu.
- 3 Befestigt mithilfe der Lüsterklemmen jeweils eine Stange mit der inneren Kupferlitze und eine Stange mit der äußeren Ummantelung.
- 4 Skizziert einen Winkel von 120° auf eurer Unterlage und klebt die Stangen entlang der Linien fest.
- 5 Schließt das andere Ende an den RTL-SDR-Stick an.

Wichtig: Ausrichtung der Antenne

Die beiden Stäbe liegen WAAGERECHT in einer Ebene und zeigen entlang der Nord-Süd-Achse. Die Winkelhalbierende des V zeigt also nach Osten oder Westen.

Grund: Ein Dipol strahlt (und empfängt) nicht in Richtung seiner Stabenden – dort liegen die Nullstellen. Zeigten die Stäbe nach Osten und Westen, läge der Satellit auf seiner Nord-Süd-Bahn genau in diesen toten Winkeln.

Weiter gilt: möglichst hoch und frei aufstellen, Abstand zu Metall und zum Boden halten, Elektronik bei Regen in eine Plastiktüte.

Aufgabe A1

Die Meteor-Satelliten senden zirkular polarisiert, euer V-Dipol empfängt aber linear polarisiert. Recherchiert, wie hoch der dadurch entstehende Verlust theoretisch ist, und begründet, warum der Empfang trotzdem funktioniert.

Die Programmierung

Wichtig für die gesamte Installation

1. Der Raspberry Pi muss während der gesamten Installation mit dem Internet verbunden sein.
2. Achtet darauf, was der Pi ausgibt. Treten Errors oder Fehlercodes auf, wiederholt den jeweiligen Schritt.
3. Alle Befehle gebt ihr in der SSH-Sitzung ein, nicht auf eurem eigenen Rechner.

1) Vorbereitung: Headless-Installation

1.1 Betriebssystem aufspielen

Spielt mit dem Raspberry Pi Imager Raspberry Pi OS Lite (64-bit) auf die SD-Karte. „Lite“ bedeutet: ohne grafische Oberfläche – genau das, was wir wollen.

Öffnet vor dem Schreiben im Imager die Einstellungen (Zahnrad-Symbol) und tragt dort ein:

- Hostname, z. B. wetterpi
- SSH aktivieren, mit Passwort-Authentifizierung
- Benutzername und Passwort (notiert euch beides!)
- W-LAN-Zugangsdaten und WLAN-Land: DE
- Zeitzone: Europe/Berlin

Achtung – Benutzername

Seit Raspberry Pi OS „Bullseye“ gibt es keinen automatischen Benutzer „pi“ mehr. Alle Pfade in dieser Anleitung verwenden deshalb \$HOME bzw. ~ und funktionieren mit jedem Benutzernamen.

1.2 Verbinden

Steckt die SD-Karte in den Pi und schließt die Stromversorgung an. Wartet etwa zwei Minuten und verbindet euch dann von eurem Rechner aus:

```
ssh benutzername@wetterpi.local
```

Falls der Name nicht gefunden wird, sucht die IP-Adresse im Router-Menü und verwendet diese stattdessen.

Bringt das System auf den aktuellen Stand:

```
sudo apt update
sudo apt full-upgrade
sudo reboot
```

Nach dem Neustart verbindet ihr euch erneut per SSH.

2) Installation der Software

2.1 Treiber für den RTL-SDR-Stick

Der Treiber muss nicht mehr selbst kompiliert werden – er liegt fertig in den Paketquellen:

```
sudo apt install rtl-sdr librtlsdr-dev
```

Der Kernel bringt einen DVB-T-Treiber mit, der dem SDR-Betrieb im Weg steht. Blockiert ihn:

```
sudo nano /etc/modprobe.d/no-rtl.conf
```

Fügt die folgenden drei Zeilen ein, speichert mit Strg + O, Enter, schließt mit Strg + X:

```
blacklist dvb_usb_rtl28xxu
blacklist rtl2832
blacklist rtl2830
```

Startet danach neu (sudo reboot) und testet den Stick:

```
rtl_test -t
```

Der Stick muss als Found 1 device(s) erkannt werden. Beendet den Test mit Strg + C.

2.2 SatDump (ohne grafische Oberfläche)

SatDump ersetzt gleich vier Programme der alten Anleitung: rtl_fm, SoX, wxmap und wxtoimg. Es demoduliert, dekodiert und legt Kartenlinien über die Bilder.

Installiert zuerst die Bibliotheken. Weil wir die GUI nicht bauen, entfallen glfw, portaudio, dbus und zenity – das spart Platz und Kompilierzeit:

```
sudo apt install git build-essential cmake g++ pkgconf \
  libfftw3-dev libpng-dev libtiff-dev libjemalloc-dev \
  libcurl4-openssl-dev libsqlite3-dev libvolk-dev libnng-dev \
  libzstd-dev libhdf5-dev librtlsdr-dev libomp-dev libarmadillo-dev
```

Danach wird SatDump heruntergeladen und übersetzt. Entscheidend ist die Option -DBUILD_GUI=OFF:

```
cd ~
git clone https://github.com/SatDump/SatDump.git
cd SatDump
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=/usr \
  -DBUILD_GUI=OFF ..
make -j$(nproc)
sudo make install
```

Geduld

Auf einem Pi 5 dauert der Vorgang etwa 15 Minuten, auf einem Pi 4 etwa 30 Minuten. Auf einem Pi 3 oder älter müsst ihr make -j1 verwenden – sonst stürzt das System ab.

Tipp: Startet lange Befehle in einer screen- oder tmux-Sitzung (sudo apt install tmux). Bricht dann die SSH-Verbindung ab, läuft der Vorgang trotzdem weiter.

Prüft, ob SatDump den SDR-Stick findet:

```
satdump sdr_probe
```

In der Liste muss euer RTL-SDR mit einer ID auftauchen.

2.3 Bahnberechnung

Für die Vorhersage der Überflüge und die zeitgesteuerte Ausführung braucht ihr zwei kleine Programme:

```
sudo apt install predict at
```

predict berechnet, wann ein Satellit über euch fliegt. at ist der Auftragsplaner von Unix: Er führt einen Befehl zu einem festgelegten Zeitpunkt aus.

3) Einrichtung

3.1 Standort eintragen

predict braucht euren Standort. Legt dafür die Datei ~/.predict/predict.qth an:

```
mkdir -p ~/.predict
nano ~/.predict/predict.qth
```

Inhalt (Beispiel Papenburg) – genau vier Zeilen: Name, Breite, Länge, Höhe in Metern:

```
Kursstation
53.0776
-7.4028
5
```

Vorsicht: zwei verschiedene Vorzeichen-Konventionen!

predict zählt den Längengrad nach WESTEN positiv. Google Maps zählt nach OSTEN positiv.

Papenburg liegt bei 7,4028° OST. In predict tragt ihr deshalb -7.4028 ein.

SatDump dagegen verwendet dieselbe Konvention wie Google Maps – dort wäre es +7.4028. Verwechselt die beiden nicht, sonst rechnet predict eure Überflüge für einen Punkt im Atlantik.

3.2 Standort auch in SatDump eintragen

SatDump braucht den Standort für die Kartenlinien. Da es keine Oberfläche gibt, bearbeitet ihr die Konfigurationsdatei direkt:

```
nano ~/.config/satdump/settings.json
```

Sucht den Abschnitt „satdump_general“ und tragt dort ein (hier mit POSITIVEM Längengrad):

```
"qth_lat": 53.0776,
"qth_lon": 7.4028,
"qth_alt": 5
```

Existiert die Datei noch nicht, startet SatDump einmal kurz (satdump sdr_probe) – dabei wird sie angelegt.

3.3 Bahndaten (TLE) holen

Legt euren Arbeitsordner an und ladet die Bahnelemente von Meteor-M N2-4 (Katalognummer 59051):

```
mkdir -p ~/wetter
nano ~/wetter/tle_update.sh
```

```
#!/bin/bash
# Bahnelemente von Meteor-M N2-4 (NORAD 59051) holen
URL="https://celestrak.org/NORAD/elements/gp.php?CATNR=59051&FORMAT=tle"
curl -s "$URL" -o "$HOME/wetter/meteor.tle"

# Plausibilitätsprüfung: Datei muss 3 Zeilen haben
if [ $(wc -l < "$HOME/wetter/meteor.tle") -lt 3 ]; then
```

```
    echo "FEHLER: TLE-Datei unvollstaendig!"
    exit 1
fi
echo "TLE aktualisiert:"
head -1 "$HOME/wetter/meteor.tle"
```

Ausführbar machen und testen: `chmod +x ~/wetter/tle_update.sh && ~/wetter/tle_update.sh`

Aufgabe A2

Öffnet die Datei meteor.tle und sucht in der zweiten Zeile das Feld „Epoch“ (Stellen 19–32). Es gibt an, wann die Bahndaten gemessen wurden. Erklärt, warum die Vorhersage ungenauer wird, je älter dieser Wert ist.

4) Der erste Empfang – von Hand

Bevor ihr etwas automatisiert, empfangt ihr einen Überflug von Hand. Nur so seht ihr, was das Programm ausgibt.

4.1 Nächsten Überflug bestimmen

```
predict -t ~/wetter/meteor.tle -p "METEOR-M2 4"
```

Ihr erhaltet eine Tabelle. Die für euch wichtigen Spalten sind:

Spalte	Bedeutung
1	Unix-Zeit (Sekunden seit 1970)
2 – 4	Datum und Uhrzeit (UTC)
5	Elevation, also der Höhenwinkel über dem Horizont
6	Azimet, also die Himmelsrichtung

Die erste Zeile ist der Aufgang (AOS), die letzte der Untergang (LOS). Nutzt nur Überflüge, bei denen die Elevation irgendwo über 20° steigt – bei flacheren Überflügen ist der Weg durch die Atmosphäre zu lang.

4.2 Empfangen

Startet den folgenden Befehl kurz vor dem berechneten Aufgang. --timeout gibt die Aufnahmedauer in Sekunden an (ein Überflug dauert etwa 12–15 Minuten):

```
satdump legacy live meteor_m2-x_lrpt ~/wetter/test1 \
--source rtl_sdr --samplerate 1.024e6 \
--frequency 137.9e6 --gain 40 --dc_block \
--timeout 840
```

Option	Bedeutung
meteor_m2-x_lrpt	die Pipeline für 72k. Für 80k: meteor_m2-x_lrpt_80k
--samplerate 1.024e6	Abtastrate; reicht für LRPT aus
--frequency 137.9e6	Sendefrequenz; Reserve ist 137.1e6
--gain 40	Verstärkung. Ohne LNA fast Maximum, mit SAW-LNA ca. 25
--dc_block	entfernt die Störspitze in der Bandmitte (bei RTL-SDR nötig)
--bias	NUR hinzufügen, wenn euer LNA über das Kabel versorgt wird
--timeout 840	Aufnahmedauer in Sekunden

72k oder 80k?

Die Meteor-Satelliten senden mit einer Symbolrate von entweder 72 000 oder 80 000 Symbolen pro Sekunde. Welche gerade verwendet wird, entscheidet der Betreiber – und ändert sich gelegentlich ohne Ankündigung. Bekommt ihr keinen Lock, wechselt die Pipeline auf meteor_m2-x_lrpt_80k und probiert es beim nächsten Überflug erneut. Das ist kein Fehler eures Aufbaus.

5) Automatischer Betrieb

Jetzt automatisiert ihr den Empfang, damit der Pi über Tage hinweg selbstständig aufzeichnet. Ihr braucht dafür zwei Skripte.

5.1 Das Empfangsskript

Legt `~/wetter/empfang.sh` an:

```
#!/bin/bash
# $1 = Bezeichnung des Ueberflugs, $2 = Dauer in Sekunden
OUT="$HOME/wetter/$1"
mkdir -p "$OUT"

satdump legacy live meteor_m2-x_lrpt "$OUT" \
  --source rtlsdr --samplerate 1.024e6 \
  --frequency 137.9e6 --gain 40 --dc_block \
  --timeout "$2" >> "$HOME/wetter/log.txt" 2>&1

echo "$(date -u +%F' '%T) fertig: $1" >> "$HOME/wetter/log.txt"
```

5.2 Das Planungsskript

Legt `~/wetter/planen.sh` an. Es sucht alle Überflüge der nächsten 24 Stunden und meldet jeden brauchbaren bei at an:

```
#!/bin/bash
SAT="METEOR-M2 4"
TLE="$HOME/wetter/meteor.tle"
MINELEV=20          # Mindest-Elevation in Grad

$HOME/wetter/tle_update.sh || exit 1

# alte Auftraege loeschen
for i in $(atq | awk '{print $1}'); do atrm $i; done

jetzt=$(date +%s)
ende=$((jetzt + 86400))
t=$jetzt

while [ $t -lt $ende ]; do
  pass=$(predict -t "$TLE" -p "$SAT" $t)
  start=$(echo "$pass" | head -1 | awk '{print $1}')
  stop=$(echo "$pass" | tail -1 | awk '{print $1}')
  maxel=$(echo "$pass" | awk -v m=0 '{if($5>m)m=$5}END{printf "%d", m}')
  dauer=$((stop - start))

  if [ "$maxel" -ge "$MINELEV" ]; then
    name=$(date -u -d "@$start" +%Y%m%d-%H%M%S)
    echo "$HOME/wetter/empfang.sh $name $dauer" \
      | at -t $(date -d "@$start" +%Y%m%d%H%M) 2>/dev/null
    echo "geplant: $name Elevation ${maxel} Grad Dauer ${dauer}s"
  fi

  t=$((stop + 60))
done
```

Macht beide Skripte ausführbar und testet das Planungsskript:

```
chmod +x ~/wetter/empfang.sh ~/wetter/planen.sh
~/wetter/planen.sh
atq                # zeigt die eingeplanten Auftraege
```

5.3 Täglich automatisch planen

Damit die Planung jede Nacht neu läuft, tragt ihr sie in `cron` ein:

```
crontab -e
```

Fügt am Ende die folgende Zeile ein. Sie startet die Planung täglich um 00:01 Uhr:

```
1 0 * * * /home/BENUTZERNAME/wetter/planen.sh >> /home/BENUTZERNAME/wetter/plan.log
2>&1
```

Achtung

In der crontab müsst ihr den vollständigen Pfad ausschreiben – cron kennt die Abkürzung `~` nicht und hat auch nicht dieselben Umgebungsvariablen wie eure SSH-Sitzung. Ersetzt `BENUTZERNAME` durch euren tatsächlichen Benutzernamen (herauszufinden mit `whoami`).

5.4 Bilder abholen

Die Bilder liegen auf dem Pi. Holt sie von eurem eigenen Rechner aus ab (nicht in der SSH-Sitzung eingeben!):

```
scp -r benutzername@wetterpi.local:~/wetter/2026*/ .
```

Alternativ startet ihr auf dem Pi kurzzeitig einen kleinen Webserver und öffnet die Adresse im Browser:

```
cd ~/wetter && python3 -m http.server 8000
```

Im Browser dann `http://wetterpi.local:8000` aufrufen. Beenden mit `Strg + C`.

6) Fehlersuche ohne Bildschirm

Ohne Wasserfall müsst ihr den Empfang aus den Textausgaben beurteilen. Verfolgt die Logdatei live mit:

```
tail -f ~/wetter/log.txt
```

Ein gelungener Überflug durchläuft immer dieselben Phasen:

- 1 Der SDR wird geöffnet und die Frequenz eingestellt.
- 2 Der Demodulator meldet steigende SNR-Werte, sobald der Satellit über den Horizont kommt.
- 3 Es erscheint eine Anzeige, dass die Frames synchronisiert sind (Lock/Sync). Ab hier entstehen Bilddaten.
- 4 Nach dem Timeout wird die Verarbeitung beendet und die Bilder werden geschrieben.

Beobachtung im Log	Bedeutung	Was tun?
Fehler beim Öffnen des SDR	Stick nicht erkannt oder blockiert	rtl_test -t ausführen; no-rtl.conf prüfen
SNR bleibt die ganze Zeit nahe 0	kein Signal	Uhrzeit, TLE und Antennenausrichtung prüfen; Satellitenstatus nachsehen
SNR steigt an, aber kein Sync	falsche Symbolrate	Pipeline auf meteor_m2-x_lrpt_80k wechseln
Sync bricht ständig ab	Signal zu schwach	LNA einsetzen, Antenne höher und freier stellen
Bilder da, Kartenlinien verschoben	Uhrzeit oder TLE veraltet	TLE aktualisieren; timedatectl prüfen
at-Aufträge werden nicht ausgeführt	atd läuft nicht	sudo systemctl enable --now atd

Prüft bei Zeitproblemen zuerst, ob die Systemuhr überhaupt synchronisiert ist:

```
timedatectl
```

In der Ausgabe muss `System clock synchronized: yes` stehen. Der Raspberry Pi hat keine batteriegepufferte Uhr und holt sich die Zeit aus dem Netz – ohne Netzverbindung sind alle Überflugzeiten und alle Kartenlinien falsch.

Satellitenstatus prüfen

Es gibt keine offizielle Statusseite von Roskosmos. Verlässliche Quellen sind die Beobachtungen anderer Funkamateure:

- db.satnogs.org – zeigt, ob in den letzten 24 Stunden jemand Daten empfangen hat
 - [r/RTLSDR](https://www.reddit.com/r/RTLSDR) und [r/amateursatellites](https://www.reddit.com/r/amateursatellites) – dort wird ein Ausfall meist innerhalb von Stunden gemeldet
- Prüft das AM TAG DES VERSUCHS, nicht in der Woche davor. Ein schweigender Satellit sieht im Log genauso aus wie eine defekte Antenne.

Auswertung Teil I

Von APT zu LRPT

Die alten NOAA-Satelliten sendeten analog: Ein 2400-Hz-Ton wurde in seiner Lautstärke verändert (amplitudenmoduliert), und jeder einzelne Helligkeitswert eines Bildpunktes entsprach direkt einer Lautstärke. Man konnte das Bild buchstäblich hören – zwei Zeilen pro Sekunde, ein Ticken wie bei einem Faxgerät. Deshalb ließ sich das Signal als .wav-Datei speichern und später dekodieren.

Meteor-M sendet digital. Die Bilddaten werden als Bitfolge übertragen und auf den Träger aufmoduliert, indem die Phase der Welle zwischen vier Zuständen umgeschaltet wird (OQPSK). Ein Ton, den man abhören könnte, existiert dabei nicht mehr – das Signal klingt wie Rauschen.

	NOAA (APT)	Meteor-M (LRPT)
Verfahren	analog, amplitudenmoduliert	digital, phasenmoduliert (OQPSK)
Datenrate	2 Bildzeilen pro Sekunde	72 000 bzw. 80 000 Symbole/s
Kanäle im Bild	2 (nebeneinander)	3 aus 6 wählbaren
Auflösung	4 km/Pixel	1 km/Pixel
Bei schwachem Signal	Bild wird verrauscht, bleibt lesbar	Bild bricht komplett ab
Speicherbar als	.wav (Audio)	.cadu (Datenrahmen)

Aufgabe A3

Vergleicht in der Tabelle die Zeile „Bei schwachem Signal“. Erklärt, warum ein analoges Verfahren allmählich schlechter wird, ein digitales dagegen plötzlich ganz ausfällt. Wo begegnet euch dieses Verhalten im Alltag?

Aufgabe A4

Meteor-M N2-4 umkreist die Erde in ca. 820 km Höhe mit einer Umlaufzeit von etwa 101 Minuten. Berechnet, um wie viel Grad sich die Erde in dieser Zeit weiterdreht, und erklärt damit, warum aufeinanderfolgende Überflüge nach Westen wandern.

Auswertung Teil II

Das Instrument MSU-MR

Die Bilder entstehen im Radiometer MSU-MR an Bord des Satelliten. Ein rotierender Spiegel tastet die Erde quer zur Flugrichtung ab und überstreicht dabei einen Streifen von etwa 2800 km Breite. Die Vorwärtsbewegung des Satelliten liefert die zweite Bilddimension – das Bild entsteht also Zeile für Zeile, genau wie bei einem Scanner.

Das Instrument misst in sechs Spektralkanälen, von denen drei gleichzeitig übertragen werden:

Kanal	Wellenlänge	Was man sieht
1	0,5 – 0,7 μm	sichtbares Licht – wie ein Foto, nur am Tag
2	0,7 – 1,1 μm	nahes Infrarot – Vegetation erscheint hell
3	1,6 – 1,8 μm	kurzwelliges IR – unterscheidet Wolken von Schnee
4	3,5 – 4,1 μm	mittleres IR – Wärmestrahlung, auch nachts
5	10,5 – 11,5 μm	thermisches IR – Temperaturmessung
6	11,5 – 12,5 μm	thermisches IR – Temperaturmessung

SatDump legt die Ergebnisse im Ausgabeordner ab. Verschafft euch zunächst einen Überblick:

```
ls -R ~/wetter/2026*/
```

Ihr findet dort unter anderem:

```
MSU-MR-1.png      <-- Kanal 1, sichtbares Licht (Graustufen)
MSU-MR-4.png      <-- Kanal 4, Infrarot (Graustufen)
MSU-MR-221.png    <-- Falschfarben aus mehreren Kanälen
MSU-MR-rgb_*.png  <-- weitere Farbkombinationen
*.cadu            <-- die rohen Datenrahmen
dataset.json       <-- Zeitstempel und Satellitenangaben
```

Aufgabe A5

Öffnet dasselbe Gebiet einmal in Kanal 1 (sichtbar) und einmal in Kanal 4 (Infrarot). Beschreibt drei Unterschiede. Welche Wolken sind nur in einem der beiden Bilder zu erkennen – und warum?

Aufgabe A6

Im Infrarotbild erscheinen hohe Wolken hell und tiefe Wolken dunkel, obwohl beide aus Wasser bestehen. Begründet dies mithilfe der Temperaturabnahme in der Atmosphäre (ca. 6,5 K pro km) und dem Stefan-Boltzmann-Gesetz.

Abschlussaufgabe

Öffnet den Wetterbericht zum Aufnahmezeitpunkt eures Satellitenbildes (in dataset.json findet ihr die genaue Zeit) und vergleicht beides miteinander. Stimmt der Wetterbericht mit der empfangenen Wolkenverteilung überein? Wo weicht er ab, und woran könnte das liegen?
